SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA04**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance

---

QUESTIONS: 5 **Total Marks: 100**

Annexure A

Analytical Problem Solving

Code of Conduct:

I S,MAHALAKSHMI(192524372)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S.MAHALAKSHMI

CO-2 (AT-3)

Analytical Problem Solving

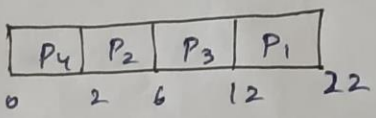
1) CPU Scheduling - Waiting Time & TAT Analysis

Process	BT	CT	AT	TAT	WT
P1	10	22	0	22	12
P2	4	6	0	6	2
P3	6	12	0	12	6
P4	2	2	0	2	0

Shortest job first

Execution order (SJF)

$P_4 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$



a) Waiting Time and TAT for each process using (SJF)

P	WT	TAT
P1	12	22
P2	2	6
P3	6	12
P4	0	2

b) Avg waiting Time & TAT

Avg WT = 12+12+6/4 = 10.5 ms

Avg TAT = 22+6+12+2/4 = 5 ms

c) Interpretation:-

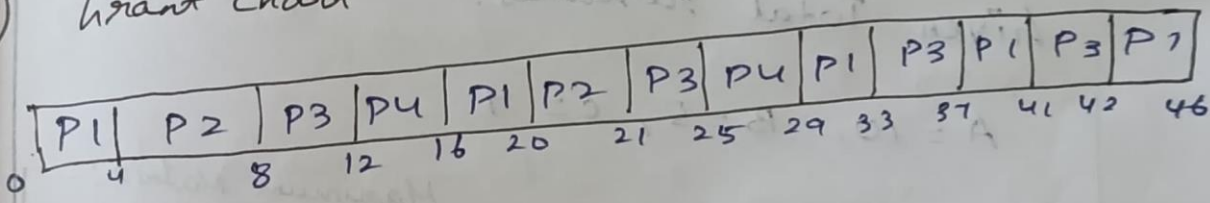
*) SJF eliminates the convoy effect caused by long process like P1.

*) It Drastically reduces the Avg WT(5ms) compared to FCFS, which improves system responsiveness and overall throughput.

2) Round Robin Scheduling - Time Quantum Impact.
(q = 4ms)

Process	BT(ms)	AT(ms)
P1	20	0
P2	5	0
P3	13	0
P4	8	0

a) Gnant Chart



b) Calculation Table:-

Process	BT	AT	TAT	CT	WT
P1	20	0	46	46	26
P2	5	0	21	21	16
P3	13	0	42	42	29
P4	8	0	29	29	21

c) Impact Analysis :-

*) Context Switching :-

Reduces Total Context switches from 12 down to 5, significantly cutting down operating system scheduling overhead

*) Response Time :-

Short Process may wait longer for their first CPU Turn.

*) Fairness & responsiveness :- A larger Quantum make Round robin behave more like FCFS, reducing responsiveness for interactive Process.

3) Dead lock - safety check using Banker's Algorithm.

Given Total resources :

A = 10, B = 5, C = 7

Allocation matrix :-

Process	A	B	C
P1	0	1	0
P2	2	0	0
P3	3	0	2
P4	2	1	1
P5	0	0	2

Maximum Matrix :-

Process	A	B	C
P1	7	5	3
P2	3	2	2
P3	9	0	2
P4	4	2	2
P5	5	3	3

Calculate the Need Matrix:-

Need = Maximum − Allocation

P_1 : (7, 5, 3) − (0, 1, 0) = (7, 4, 3)

P_2 : (3, 2, 2) − (2, 0, 0) = (1, 2, 2)

P_3 : (9, 0, 2) − (3, 0, 2) = (6, 0, 0)

P_4 : (4, 2, 2) − (2, 1, 1) = (2, 1, 1)

P_5 : (5, 3, 3) − (0, 0, 2) = (5, 3, 1)

Need Matrix:-

Process	A	B	C
P1	7	4	3
P2	1	2	2
P3	6	0	0
P4	2	1	1
P5	5	3	1

Calculate the Available Resources

Available = Total − Allocated

Total Allocated resources:-

A = 0 + 2 + 3 + 2 + 0 = 7

B = 1 + 0 + 0 + 1 + 0 = 2

C = 0 + 0 + 2 + 1 + 2 = 5

∴ Available = (10, 5, 7) − (7, 2, 5)

Available = (3, 3, 2)

Perform the safety check

A process can execute if its Need is less than or equal to the currently available resources for all resources types.

Step	Need (A, B, C)	Process	Available before execution	Available After ex
1)	(1, 2, 2)	P2	(3, 3, 2)	(5, 3, 2)
2)	(2, 1, 1)	P4	(5, 3, 2)	(7, 4, 3)
3)	(7, 4, 3)	P1	(7, 4, 3)	(7, 5, 3)
4)	(6, 0, 0)	P3	(7, 5, 3)	(10, 5, 5)
5)	(5, 3, 1)	P5	(10, 5, 5)	(10, 5, 5)

Determine the State state

Every process can finish successfully.

Safe Sequence: $P_2 \rightarrow P_4 \rightarrow P_1 \rightarrow P_3 \rightarrow P_5$

4) Deadlock Dedication - Resources Allocation - Graph (RAG)

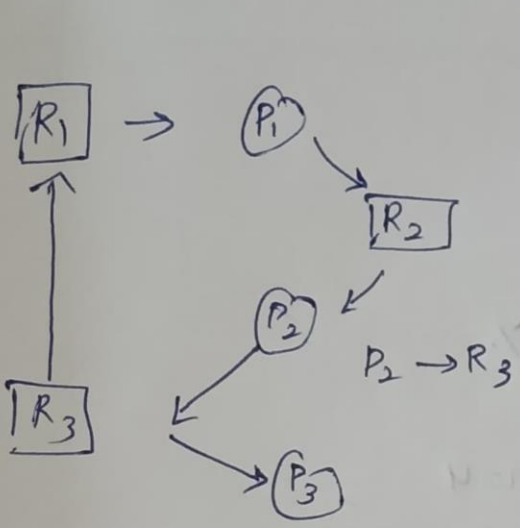
a) The representation of Resource Allocation Graph.

*) Process - Circle
*) Resource - Square

*) Resource Allocation
- Edge goes from resource to process.

*) A request edge goes from process to resource

GRAPH



Cycle: $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$

Allocation edges :- $R_1 \rightarrow P_1$, $R_2 \rightarrow P_2$, $R_3 \rightarrow P_3$.

Request edges :- $P_1 \rightarrow R_2$, $P_2 \rightarrow R_3$, $P_3 \rightarrow R_1$

Thus, the graph forms a cycle.

$P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$

b) Determine whether deadlock exits :-

Yes, Deadlock exists. Each process holds one resources and wait for another resource held by the next process. This creates a circular wait, assuming each resource has one instance.

c) Strategy to resolve deadlock

⇒ Select a process as the victim
⇒ Preempt its resource, if possible
⇒ Allocate the released resource to a waiting process
⇒ Allow process to complete and release resources.

5) CPU utilization:-

Given

CPU Time = 20%.
I/O WT = 80%.
No. of Process ⇒ n=4
Prob of I/O W ⇒ P=0.8

a) CPU utilization for 4 Process

$U = 1 - P^n$
$U = 1 - (0.8)^4$
$U = 1 - 0.4096$
$U = 0.5904 \Rightarrow (59.04\%)$

b) Calculate CPU utilization for 6 process

$U = 1 - (0.8)^6$
$= 1 - 0.262144$
$U = 0.7378$
$U = 73.79\%$

c) Interpretation.

Process	CPU utilization
4	59.64 %.
6	73.79 %.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

TOTAL MARKS AWARDED **FACULTY SIGNATURE**